HAW HAMBURG

BACHELORTHESIS

Philipp Günther

Automatische Detektion und 3D-Lokalisierung von Obstbäumen im Rahmen des SAMSON-Projekts

FAKULTÄT TECHNIK UND INFORMATIK
Department Informatik

Faculty of Computer Science and Engineering
Department Computer Science

HOCHSCHULE FÜR ANGEWANDTE WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

Philipp Günther

Automatische Detektion und 3D-Lokalisierung von Obstbäumen im Rahmen des SAMSON-Projekts

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Bachelor of Science Informatik Technischer Systeme*
am Department Informatik
der Fakultät Technik und Informatik
der Hochschule für Angewandte Wissenschaften Hamburg

Betreuender Prüfer: Prof. Dr. Peer Stelldinger
Zweitgutachter: Prof. Dr. Jörg Dahlkemper

Eingereicht am: 16.03.2026

Philipp Günther

Thema der Arbeit

Automatische Detektion und 3D-Lokalisierung von Obstbäumen im Rahmen des SAMSON-Projekts

Stichworte

Smart Farming, Agrarrobotik, Instance Segmentation, Georeferenzierung, Sensorfusion, YOLOv8

Kurzzusammenfassung

Im Rahmen dieser Bachelorarbeit wird eine Software-Pipeline entwickelt, um temporäre 2D-Bilddaten eines autonomen Agrarroboters (SAMSON) in globale 3D-Geodaten zu überführen. Ziel ist die automatisierte Erkennung und exakte Georeferenzierung von Obstbäumen für Smart-Farming-Anwendungen.

Die methodische Basis bildet ein YOLOv8-seg-Modell, das mithilfe von SAM 2.0 auf die Segmentierung von Baumstämmen trainiert wurde. Um die 2D-Pixelkoordinaten in den 3D-Raum zu projizieren, wird der Fußpunkt der detektierten Bäume über einen Median-Filter mit DVSO-Tiefendaten fusioniert. Anschließend erfolgt die globale Verortung im WGS84-Referenzsystem durch Verknüpfung mit den hochpräzisen RTK-GNSS-Odometriedaten des Roboters.

Die Evaluierung belegt die Robustheit der Pipeline gegenüber landwirtschaftlichen Störfaktoren. Schwächen in der reinen 2D-Erkennung (mAP50 von 49,1 %) werden durch algorithmische Tiefenfilter und temporale Frame-Skip-Logik erfolgreich kompensiert. Das resultierende Endprodukt – eine GIS-kompatible CSV-Datei – erbringt den erfolgreichen Machbarkeitsnachweis (Proof of Concept) für die KI-gestützte Einzelbaumkartierung

Philipp Günther

Title of Thesis

Automated Detection and 3D Localization of Fruit Trees within the SAMSON Project

Keywords

Smart Farming, Agricultural Robotics, Instance Segmentation, Georeferencing, Sensor Fusion, YOLOv8

Abstract

In the context of this bachelor's thesis, a software pipeline is developed to transform temporary 2D image data from an autonomous agricultural robot (SAMSON) into global 3D spatial data. The objective is the automated detection and exact georeferencing of fruit trees for smart farming applications.

The methodological basis is a YOLOv8-seg model trained for tree trunk segmentation using SAM 2.0. To project the 2D pixel coordinates into 3D space, the base point of the detected trees is fused with DVSO depth data via a median filter. Subsequently, global localization in the WGS84 reference system is achieved by linking these coordinates with the high-precision RTK-GNSS odometry data of the robot.

The evaluation demonstrates the robustness of the pipeline against agricultural interference factors. Weaknesses in the isolated 2D detection (mAP50 of 49.1%) are successfully compensated for by algorithmic depth filters and temporal frame-skip logic. The resulting final product—a GIS-compatible CSV file—successfully provides the proof of concept for AI-assisted individual tree mapping

Inhaltsverzeichnis

Abbildungsverzeichnis ..... **vii**

1 Einleitung **1**

1.1 Motivation und Ausgangslage 1

1.2 Problemstellung 1

1.3 Zielsetzung der Arbeit 2

1.4 Aufbau der Arbeit 3

2 Theoretische Grundlagen **4**

2.1 Objekterkennung und Instance Segmentation 4

2.2 Die YOLO-Architektur (You Only Look Once) 4

2.3 Tiefenwahrnehmung und DVSO (Direct Visual Stereo Odometry) 5

2.4 Kinematische Transformationen und Koordinatensysteme 6

3 Methodik und Implementierung **7**

3.1 Datengrundlage und Sensor-Setup 7

3.2 Datenannotation und KI-gestützte Segmentierung 9

3.3 Modellkonfiguration und Training 11

3.4 Übersicht der Verarbeitungspipeline 13

3.5 Datenfusion und räumliche Vorverarbeitung 16

3.6 2D-Objekterkennung und Segment-Filterung 17

3.7 3D-Rekonstruktion und Tiefenbestimmung 18

3.8 Globale Georeferenzierung und Datenpersistierung 20

4 Ergebnisse und Evaluation **23**

4.1 Evaluationsmetriken der Objekterkennung 23

4.2 Trainingsverlauf und Modellgüte 25

4.3 Architekturvergleich: Genauigkeit versus Echtzeitfähigkeit 30

4.4 Qualitative Objekterkennung und visuelle Evaluierung 31
4.5 Ergebnisse der Georeferenzierung und Kartierung 37
5 Fazit und Ausblick 40
5.1 Zusammenfassung und Fazit 40
5.2 Ausblick 41
Literaturverzeichnis 42
A Anhang 46
A.1 Verwendete Hilfsmittel 46

Abbildungsverzeichnis

Abbildung 1:Benutzeroberfläche des CVAT-Annotationstools während der KI-gestützten Segmentierung eines Baumstammes. Durch interaktive Punkt-Prompts (positive/negative Marker) generiert das integrierte SAM 2.0-Modell automatisch die polygonale Objektmaske. Quelle: Eigene Darstellung 10

Abbildung 2: Schematische Darstellung der entwickelten Verarbeitungspipeline. Der Prozess umfasst das Einlesen der Sensor-daten, die 2D-Objekterkennung mittels YOLO, die Extraktion der Tiefe sowie die Transformation der lokalen 3D-Koordinaten in globale GPS-Längen und Breitengrade. Quelle: Eigene Darstellung 15

Abbildung 3:Entwicklung der Fehlerfunktionen (Loss) auf dem Trainingsdatensatz über 83 Epochen. Der steile initiale Abfall belegt einen schnellen Lernfortschritt. Quelle: Eigene Darstellung 25

Abbildung 4: Entwicklung der Fehlerfunktion auf dem Validierungsdatensatz. Das Erreichen des globalen Minimums ist erkennbar. Quelle: Eigene Darstellung 26

Abbildung 5:Verlauf der Mean Average Precision (mAP50) für Bounding-Boxen und Segmentierungsmasken. Nach einem anfänglich starken Anstieg konvergieren die Werte ab Epoche 30 asymptotisch gegen ihr Maximum. Quelle: Eigene Darstellung 27

Abbildung 6: Confusion Matrix. Quelle: Eigene Darstellung 28

Abbildung 7: Inferenz des YOLOv8-seg-Modells unter Standardbedingungen. Der Grüne Kasten stellt die Bounding-Box da und der rote Punkt den tiefsten Punkt der Segmentierungsmaske. Quelle: Eigene Darstellung 32

Abbildung 8: Objekterkennung bei visuellen Verdeckungen und hartem Schattenwurf. Trotz der teilweisen Fragmentierung des Stammes durch Blattwerk bleibt der relevante untere Stammbereich für die Tiefenextraktion detektierbar. Quelle: Eigene Darstellung 34

Abbildung 9: Typisches Fehlerbild in der Praxis. Baumstamm wurde aufgrund Verdeckung durch Blätter und Blüten nicht erkannt. Quelle: Eigene Darstellung 36

Abbildung 10: Visualisierung der generierten GPS-Koordinaten auf digitalem Kartenmaterial (QGIS). Die Baumreihen der Plantage sind deutlich erkennbar. 38

1 Einleitung

1.1 Motivation und Ausgangslage

Die moderne Landwirtschaft steht im 21. Jahrhundert vor immensen ökologischen und ökonomischen Herausforderungen. Um steigende Ertragsanforderungen mit dem schonenden Einsatz von Ressourcen, wie Wasser, Düngemitteln und Pflanzenschutzmitteln, in Einklang zu bringen, etabliert sich zunehmend das Konzept der Präzisionslandwirtschaft (*Precision Agriculture* bzw. *Smart Farming*) (Walter et al., 2017). Ein zentraler Baustein dieser Entwicklung ist der Einsatz autonomer Agrarroboter (Shamshiri et al., 2018). Diese Systeme können landwirtschaftliche Flächen nicht nur automatisiert bewirtschaften, sondern diese auch sensorisch erfassen und digitalisieren.

Besonders in Sonderkulturen wie Obstanlagen (Obstplantagen) ist eine feingranulare, baumgenaue Bewirtschaftung von großem Interesse. Um Pflegemaßnahmen wie den selektiven Baumschnitt, die gezielte Applikation von Nährstoffen oder eine automatisierte Ernte durchführen zu können, benötigt ein autonomes System zwingend eine hochpräzise digitale Karte seiner Umgebung (Bargoti & Underwood, 2017). Die Grundlage einer solchen Karte ist das Wissen über die exakte globale Position jedes einzelnen Baumes.

1.2 Problemstellung

Die manuelle geodätische Vermessung von Tausenden Bäumen in einer Plantage ist zeit- und kostenintensiv (Underwood et al., 2015). Zwar navigieren Feldroboter wie der in dieser Arbeit verwendete SAMSON-Roboter (Blome et al., 2024) bereits mit hochpräzisen GPS- und

Odometriesystemen, jedoch erfassen diese Systeme primär die Position des Roboters selbst und nicht die der umgebenden Pflanzen.

Die automatisierte Erfassung der Bäume während der Vorbeifahrt stellt eine komplexe technische Herausforderung dar. Die landwirtschaftliche Umgebung ist hochgradig unstrukturiert: Wechselnde Lichtverhältnisse, harter Schattenwurf, komplexe Aststrukturen sowie Verdeckungen durch künstliche Stützpfosten und Blattwerk erschweren die zuverlässige visuelle Objekterkennung (Gongal et al., 2015). Zudem reicht eine rein zweidimensionale Erkennung im Kamerabild nicht aus. Um einen Baum auf einer Landkarte zu verorten, muss seine relative Distanz zum Roboter berechnet und anschließend durch kinematische Transformationen in das globale Geokoordinatensystem überführt werden (Siegwart et al., 2011).

1.3 Zielsetzung der Arbeit

Das primäre Ziel dieser Bachelorarbeit ist die Entwicklung und Evaluierung einer robusten, mehrstufigen Software-Pipeline, die in der Lage ist, aus den multimodalen Sensordaten eines navigierenden Agrarroboters vollautomatisiert eine globale digitale Karte der umgebenden Baumstämme zu generieren.

Zur Erreichung dieses Ziels werden folgende ingenieurswissenschaftliche Teilaufgaben adressiert:

1. **KI-basierte Objekterkennung:** Das Training eines State-of-the-Art neuronalen Netzes (YOLOv8-seg) (Jocher et al., 2023) zur pixelgenauen Segmentierung von Baumstämmen unter erschwerten landwirtschaftlichen Bedingungen.
2. **Algorithmische Filterung:** Die Entwicklung von Post-Processing-Algorithmen zur Bereinigung von Mehrfacherkennungen und zur algorithmischen Bestimmung des untersten sichtbaren Stammpunktes innerhalb des Kamerasichtfeldes, welcher für die weitere Projektion als Referenz-Fußpunkt fungiert.
3. **3D-Tiefenextraktion:** Die Verknüpfung der 2D-Bilddaten mit dichten DVSO-Tiefenkarten (Wang et al., 2017) zur Berechnung lokaler Raumkoordinaten.

4. **Globale Georeferenzierung:** Die mathematische Überführung der lokalen Koordinaten in das standardisierte WGS84-System (Bill, 2016) mittels der aufgezeichneten Roboter-Odometrie.

Das finale Resultat der Pipeline soll eine maschinenlesbare Geodaten-Tabelle (CSV) sein, welche die GPS-Koordinaten aller erfolgreich detektierten Bäume persistiert und somit direkt in etablierte Geoinformationssysteme (GIS) importiert werden kann.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in fünf Hauptkapitel. Nach der Einleitung in Kapitel 1 werden in Kapitel 2 die theoretischen und technologischen Grundlagen erläutert, die für das Verständnis der entwickelten Pipeline essenziell sind. Dies umfasst die Konzepte der Instance Segmentation (YOLO) sowie die Prinzipien der räumlichen Transformationen in der Robotik. Kapitel 3 bildet das methodische Herzstück der Arbeit und beschreibt detailliert die Konzeption, das Datensatz-Training sowie den algorithmischen Aufbau der Georeferenzierungs-Pipeline. In Kapitel 4 werden die Ergebnisse der Pipeline quantitativ und qualitativ evaluiert, wobei insbesondere auf die Herausforderungen der Objekterkennung in unstrukturierten Umgebungen eingegangen wird. Die Arbeit schließt in Kapitel 5 mit einer Zusammenfassung der gewonnenen Erkenntnisse und einem Ausblick auf zukünftige Optimierungspotenziale.

2 Theoretische Grundlagen

Um die in dieser Arbeit entwickelte Methodik zur automatisierten Baumkartierung einordnen zu können, werden in diesem Kapitel die wesentlichen technologischen und mathematischen Konzepte erläutert. Dies umfasst die Grundlagen der modernen Bildverarbeitung sowie die Prinzipien der räumlichen Tiefenwahrnehmung und Kinematik.

2.1 Objekterkennung und Instance Segmentation

Die maschinelle visuelle Wahrnehmung (Computer Vision) klassifiziert Erkennungsaufgaben in verschiedene Detailgrade. Die klassische *Object Detection* (Objekterkennung) lokalisiert Objekte in einem Bild grob mittels rechteckiger Begrenzungsrahmen, sogenannter *Bounding-Boxen* (Szeliski, 2022).

Für präzise landwirtschaftliche Anwendungen, bei denen die exakte Form und der Bodenkontaktpunkt eines Objektes (z. B. eines Baumstammes) von Bedeutung sind, reicht eine Bounding-Box jedoch oft nicht aus. Hier kommt die *Instance Segmentation* zum Einsatz. Diese Technik kombiniert die Objekterkennung mit der semantischen Segmentierung (He et al., 2017). Sie identifiziert nicht nur die Klasse und den Ort eines Objektes, sondern generiert für jede einzelne Objektinstanz eine pixelgenaue, polygonale Maske, welche das Objekt exakt vom Hintergrund isoliert.

2.2 Die YOLO-Architektur (You Only Look Once)

Für die Echtzeit-Objekterkennung hat sich die YOLO-Modellfamilie (You Only Look Once) als De-facto-Standard etabliert (Redmon et al., 2016). Im Gegensatz zu zweistufigen Detektoren wie Faster R-CNN (Ren et al., 2015), die zunächst Regionen vorschlagen und diese anschließend klassifizieren, betrachtet YOLO das gesamte Bild in einem einzigen Durchlauf

(Single-Stage-Detector). Das Bild wird in ein Raster unterteilt, und ein Convolutional Neural Network (CNN) sagt simultan Bounding-Boxen, Klassenwahrscheinlichkeiten und Konfidenzwerte vorher. Dies ermöglicht schnelle Inferenzzeiten, die für den Einsatz auf mobilen Robotern zwingend erforderlich sind.

In dieser Arbeit wird die Modellvariante **YOLOv8-seg** von Ultralytics (Jocher, et al., 2023) verwendet. Diese Architektur erweitert den klassischen YOLO-Detektor um einen zusätzlichen Segmentierungskopf (Segmentation Head), der parallel zur Bounding-Box auch die Koeffizienten für die Instance-Segmentation-Masken ausgibt.

2.3 Tiefenwahrnehmung und DVSO (Direct Visual Stereo Odometry)

Um Objekte aus einem 2D-Bild in den 3D-Raum zu projizieren, ist die Kenntnis über die Distanz (Tiefe) zwingend erforderlich. Der in dieser Arbeit verwendete SAMSON-Roboter nutzt hierfür ein Stereo-Kamerasystem. Ähnlich dem menschlichen Sehen berechnen Stereo-Kameras die Tiefe (Z) eines Pixels durch die Disparität (Verschiebung) desselben Objektes zwischen dem linken und rechten Kamerabild (Triangulation) (Hartley & Zisserman, 2004).

Diese Tiefendaten fließen in die DVSO ein. Odometrie bezeichnet in der Robotik die fortlaufende Schätzung der eigenen Position und Ausrichtung (Pose) durch die Auswertung von Sensordaten (Siegwart et al., 2011). Die DVSO analysiert die zeitliche Veränderung von Pixelintensitäten über aufeinanderfolgende Bilder hinweg und kombiniert diese mit den berechneten Tiefeninformationen (Wang et al., 2017). Das Resultat dieses Systems sind zwei für diese Arbeit essenzielle Datenstrukturen:

1. **Die Tiefenkarte** (Depth Map): Eine Matrix, bei der jeder Pixelwert die physikalische Distanz zum Kameraobjektiv in Metern repräsentiert.
2. **Die Trajektorien-Matrix** (T_{world_cam}): Eine kontinuierlich aktualisierte Transformationsmatrix, welche die genaue globale Position des Roboters zum Zeitpunkt der Bildaufnahme beschreibt.

2.4 Kinematische Transformationen und Koordinatensysteme

Um lokale Messwerte (wie den Abstand zu einem Baum) in eine globale Landkarte (WGS84-GPS-Koordinaten) einzutragen, müssen die Daten verschiedene Koordinatensysteme durchlaufen. In der Robotik wird dies durch Matrixmultiplikationen mit sogenannten homogenen Transformationsmatrizen (4x4-Matrizen) gelöst (Craig, 2018).

Eine homogene Matrix vereint dabei zwei mathematische Operationen: Eine 3x3-Rotationsmatrix zur Änderung der Ausrichtung (z. B. der Wechsel vom optischen Kamera-System in das Navigations-System des Roboters) und einen 3x1-Translationsvektor für die räumliche Verschiebung in Metern. Durch die Erweiterung von 3D-Vektoren um eine vierte Dimension (die homogene Konstante 1) können Translationen und Rotationen elegant und ressourcenschonend in einem einzigen Multiplikationsschritt auf der Hardware (CPU/GPU) berechnet werde

3 Methodik und Implementierung

3.1 Datengrundlage und Sensor-Setup

Um die entwickelte Georeferenzierungs-Pipeline unter realen landwirtschaftlichen Bedingungen zu trainieren und zu validieren, wurde ein originärer Datensatz herangezogen. Die Aufzeichnung der multimodalen Sensordaten erfolgte durch den navigierenden SAMSON-Roboter auf einer kommerziell bewirtschafteten Obstanlage in Estebrügge in der Region Altes Land, einem der größten geschlossenen Obstanbaugebiete Nordeuropas.

Optische Sensorik und phänologischer Zustand: Als primärer optischer Sensor für die Bildverarbeitung diente eine hochauflösende, industrielle Flächenkamera des Typs *Basler a2A5328-15ucPro* (Basler AG, 2023), welche starr in das Sensorsystem des Roboters integriert war. Zum Zeitpunkt der Datenerfassung befanden sich die Obstbäume in einer frühen phänologischen Phase, die nach der standardisierten BBCH-Skala den Stadien vom Knospenschwellen bis zur beginnenden Blüte zuzuordnen ist (Meier, 2001). Das weitgehende Fehlen eines dichten Blätterdachs definiert maßgeblich die visuelle Komplexität dieses Datensatzes: Einerseits treten weniger Verdeckungen (Occlusions) im direkten Vordergrund auf, andererseits erhöht sich die Sichttiefe in die Plantage drastisch. Dies erzwingt, dass das neuronale Netz relevante Baumstämme im Vordergrund präzise von visuell ähnlichen Stämmen in den weiter entfernten, parallel verlaufenden Baumreihen separieren muss.

Datenaufbereitung und Frame-Extraktion: Während der Fahrt zeichnete das System einen kontinuierlichen Datenstrom auf, der zunächst in proprietären `.rec`-Dateien (Recording-Containern) persistiert wurde. Um aus diesem massiven, zeitlich stark korrelierten Videostrom einen informationsteiligen Datensatz für das Training des YOLO-Modells zu generieren, wurde ein automatisiertes temporales Sub-Sampling durchgeführt. Aus den Rohdateien

wurde systematisch nur jedes dritte Bild (Frame) extrahiert. Diese Reduktion garantiert eine ausreichend hohe visuelle Varianz durch die fortschreitende Eigenbewegung des Roboters, verhindert jedoch ein Überlernen (Overfitting) des Modells durch nahezu identische, aufeinanderfolgende Bilder. Aus dem resultierenden Pool an extrahierten Bildern wurden schließlich 1014 repräsentative Aufnahmen für die manuelle Polygon-Annotation (vgl. Kapitel 3.2) selektiert.

Globale Positionierung und Odometrie: Um die in den Bilddaten detektierten Bäume in das globale WGS84-Referenzsystem überführen zu können, wurde die Trajektorie des Roboters kontinuierlich erfasst. Die absolute globale Verortung erfolgte hierbei über ein hochpräzises RTK-GNSS-System (Real-Time Kinematic Global Navigation Satellite System). Nach initialen Tests wurde für die finale Datenerhebung das fortschrittliche *simpleRTK3B Heading*-Board des Herstellers Ardusimple in das Sensorsystem des Roboters integriert, welches auf dem *mosaic-H*-Chip von Septentrio basiert (ArduSimple, 2023).

Ein entscheidender technologischer Vorteil dieses Systems ist die Verwendung von zwei Multiband-GNSS-Antennen. Diese Dual-Antennen-Konfiguration ermöglicht nicht nur eine zentimetergenaue Positionsbestimmung, sondern liefert simultan eine hochpräzise Ausrichtung (Heading/Orientierung) des Roboters bei einer hohen Aktualisierungsrate von 50 Hz. Um die für die landwirtschaftliche Einzelbaumkartierung zwingend erforderliche Zentimetergenauigkeit zu gewährleisten, bezog das System kontinuierlich kinematische Echtzeit-Korrekturdaten über den SAPOS-Dienst (Satellitenpositionierungsdienst) des Landes Niedersachsen (Landesamt für Geoinformation und Landesvermessung Niedersachsen (LGLN), 2026).

Die Hardware-Kommunikation sowie die Stromversorgung erfolgten via USB-C über den zentralen Verarbeitungsrechner des Roboters (Nvidia Orin). Softwareseitig wurde der Datenstrom nahtlos über den C++-basierten *ROSaic*-Treiber in die bestehende ROS-Umgebung (Robot Operating System, Distribution *Noetic*) integriert (Quigley et al., 2009). Die daraus resultierenden, hochfrequenten Positions- und Orientierungsdaten lagen synchronisiert zu den optischen Bilddaten vor und bildeten das fundamentale geometrische Gerüst für die exakte Konstruktion der Trajektorien-Matrix (T_{world_cam}) in der nachgelagerten 3D-Georeferenzierung.

3.2 Datenannotation und KI-gestützte Segmentierung

Um das YOLO-Modell auf die spezifische Erkennung von Baumstämmen (Klasse `trunk`) im landwirtschaftlichen Umfeld zu trainieren, war die Erstellung eines hochwertigen Ground-Truth-Datensatzes erforderlich. Da die entwickelte Pipeline zur Tiefenbestimmung auf dem untersten, von der Kamera visuell erfassten Punkt des Stammes (sichtbarer Fußpunkt) basiert (vgl. Kapitel 3.4), reichten einfache Bounding-Boxen als Trainingsdaten nicht aus. Stattdessen mussten pixelgenaue Polygon-Masken (Instance Segmentation) für jeden sichtbaren Baumstamm erstellt werden.

Für diesen Annotationsprozess wurde die webbasierte Open-Source-Plattform CVAT (Computer Vision Annotation Tool) eingesetzt (Sekachev et al., 2020). Um den zeitintensiven Prozess der manuellen Polygon-Erstellung zu optimieren und gleichzeitig die Genauigkeit der Maskenränder an den unregelmäßigen Baumstrukturen zu maximieren, wurde ein KI-gestützter Annotations-Workflow (AI-assisted Labeling) genutzt. Hierfür kam das *Segment Anything Model 2* (SAM 2.0) zum Einsatz (Ravi et al., 2024), welches direkt als integriertes Werkzeug in der CVAT-Browseranwendung zur Verfügung steht.

Anstatt die Objektkonturen manuell Punkt für Punkt nachzuzeichnen, konnte die Segmentierung komfortabel durch interaktive Prompts direkt im Browser gesteuert werden. Durch das Setzen gezielter Klick-Punkte (positive Markierungen auf dem Zielobjekt) generierte SAM 2.0 in Echtzeit automatisch hochpräzise Segmentierungsmasken des jeweiligen Baumstammes, wie in Abbildung 1 exemplarisch dargestellt ist.

Abbildung 1:Benutzeroberfläche des CVAT-Annotationstools während der KI-gestützten Segmentierung eines Baumstammes. Durch interaktive Punkt-Prompts (positive/negative Marker) generiert das integrierte SAM 2.0-Modell automatisch die polygonale Objektmaske. Quelle: Eigene Darstellung

Um die hohe Qualität des Trainingsdatensatzes zu gewährleisten, wurde jede durch die KI generierte Maske abschließend einer manuellen Sichtprüfung unterzogen. Bei algorithmischen Fehlsegmentierungen, beispielsweise bedingt durch optische Verdeckungen durch Blattwerk, Äste oder Stützpfosten, wurden die Polygonpunkte händisch über die CVAT-Benutzeroberfläche korrigiert. Dieses hybride Vorgehen reduzierte den zeitlichen Aufwand der Datenaufbereitung erheblich und stellte gleichzeitig sicher, dass das YOLO-Modell im anschließenden Training mit hochakkuraten, pixelgenauen Objektgrenzen gespeist wurde.

3.3 Modellkonfiguration und Training

Nach der vollständigen Annotation in CVAT wurden die generierten Polygon-Daten in das benötigte YOLO-Format exportiert. Um eine objektive Evaluierung der Modellleistung zu gewährleisten und einem Auswendiglernen der Daten (Overfitting) entgegenzuwirken, wurde der Gesamtdatensatz von 1014 Bildern in ein Trainingsset und ein Validierungsset unterteilt. Es wurde ein bewährter Split von 80/20 angewendet, woraus 815 Bilder für das eigentliche Training und 199 Bilder für die Validierung während des Trainingsprozesses resultierten.

Als Basisarchitektur wurde das *YOLOv8s-seg* Modell (Small-Variante für Instance Segmentation) von Ultralytics gewählt. Diese Modellgröße stellt für den Einsatz in der mobilen Robotik einen optimalen Kompromiss dar: Sie bietet eine ausreichend hohe Kapazität für komplexe Segmentierungsaufgaben bei gleichzeitig moderaten Anforderungen an die Inferenzzeit und den Videospeicher.

Die Relevanz der Multi-Klassen-Detektion (Trunk vs. Pole): Ein essenzieller Aspekt der Datensatzgestaltung war die explizite Definition und Annotation einer zweiten Objektklasse neben den Baumstämmen (`trunk`): den künstlichen Stützpfosten (`pole`). In modernen Sonderkulturen werden Bäume häufig durch vertikale Pfosten aus Holz, Beton oder Metall gestützt. Da diese Strukturen eine starke geometrische Ähnlichkeit zu schmalen Baumstämmen aufweisen, bestünde bei einem reinen Ein-Klassen-Modell ein immenses Risiko für Falsch-Positiv-Erkennungen (False Positives) durch das neuronale Netz. Indem das Modell gezielt darauf trainiert wird, strikt zwischen `trunk` und `pole` zu differenzieren, erlernt es die subtilen

visuellen Unterschiede in Textur, Farbe und Kantengeradheit. Diese präzise Klassentrennung bildet die elementare Voraussetzung für die spätere algorithmische Datenbereinigung, bei der Pfosten konsequent aussortiert werden, um fehlerhafte Georeferenzierungen in der finalen Karte zu vermeiden.

Hyperparameter und Erhöhung der Eingangsauflösung: Ein kritischer Faktor bei der Erkennung von Obstbäumen ist deren Geometrie. Baumstämme (und eventuelle Stützpfosten) stellen im Bildraum oft sehr schmale, vertikale Strukturen dar, die bei einer starken Komprimierung der Bilddaten verschwinden können. Aus diesem Grund wurde die standardmäßige Eingangsauflösung des Modells von 640x640 Pixeln signifikant auf `imgsz = 1024` Pixel erhöht. Um den dadurch stark gestiegenen Bedarf an GPU-Speicher (VRAM) zu kompensieren, wurde die Batch-Größe (Batch-Size) auf 4 reduziert. Das Modell wurde für maximal 300 Epochen trainiert, wobei ein Early-Stopping-Mechanismus (`patience = 50`) implementiert wurde. Dieser beendet den Trainingslauf frühzeitig, sobald sich die Validierungsmetriken über 50 aufeinanderfolgende Epochen nicht mehr verbessern, was Rechenzeit spart und Overfitting verhindert.

Physikalisch motivierte Datenaugmentation Um die Robustheit des Modells gegenüber den wechselnden Umweltbedingungen im realen landwirtschaftlichen Einsatz zu maximieren, wurde während des Trainings eine gezielte, dynamische Datenaugmentation (Data Augmentation) angewendet. Dabei wurden die Parameter spezifisch an die physikalischen Gegebenheiten des SAMSON-Roboters und der Plantage angepasst:

- **Licht- und Wettervarianz:** Durch zufällige Modifikationen im HSV-Farbraum (Farbton: 1,5 %, Sättigung: 70 %, Helligkeit: 40 %) wurden verschiedene Tageszeiten, Bewölkungsgrade und Schattenwürfe simuliert.
- **Geometrische Transformationen:** Eine zufällige Skalierung (`scale = 0.6`) simulierte unterschiedliche Distanzen zu den Bäumen. Um die Erschütterungen und das Wanken des Roboters auf unebenem Agrarboden abzubilden, wurden die Bilder um bis zu ± 5 Grad rotiert.
- **Spiegelungen:** Während eine horizontale Spiegelung (`fliplr = 0.5`) zur Verdopplung der geometrischen Varianz genutzt wurde, wurde eine vertikale Spiegelung

(`flipud = 0.0`) explizit deaktiviert, da dies der natürlichen Wachstumsausrichtung von Bäumen (Gravitropismus) physikalisch widersprechen würde.

3.4 Übersicht der Verarbeitungspipeline

Um aus den rohen, zweidimensionalen Kameradaten eines navigierenden Agrarroboters präzise globale GPS-Koordinaten (Längen- und Breitengrad) der erfassten Obstbäume zu generieren, wurde im Rahmen dieser Arbeit eine mehrstufige Software-Pipeline in Python entwickelt. Die zentrale Herausforderung hierbei besteht darin, die semantische Information aus dem Bild (das erkannte Objekt "Baumstamm") über mehrere Koordinatensysteme hinweg in die reale Welt zu transformieren.

Dieser komplexe Prozess lässt sich grob in vier aufeinanderfolgende Phasen unterteilen, die in Abbildung 2 schematisch dargestellt sind:

1. **Datenfusion und Vorverarbeitung:** Im ersten Schritt werden die synchronisierten Datenströme des Roboters eingelesen. Dies umfasst die hochauflösenden RGB-Bilder einer Kamera, die zugehörigen Tiefenkarten aus der DVSO-Berechnung sowie die lokalen Trajektoriedaten, welche die exakte Pose des Roboters zum Zeitpunkt der Aufnahme beschreiben.
2. **2D-Objekterkennung und Segmentierung:** Auf den vorverarbeiteten Bilddaten wird das vortrainierte YOLO-Modell angewendet. Es identifiziert die sichtbaren Baumstämme und generiert Pixelmasken. Durch einen eigens entwickelten Filteralgorithmus werden fragmentierte Stammsegmente zusammengefasst und der tiefste von der Kamera visuell erfasste Punkt des Baumes (sichtbarer Fußpunkt) im 2D-Bildraum isoliert.
3. **Lokale 3D-Rückprojektion:** Da ein 2D-Pixel allein keine Entfernungsinformation enthält, wird der zugehörige Distanzwert am Fußpunkt aus der DVSO-Tiefenkarte extrahiert. Um Messfehler abzufangen, kommt hierbei ein lokaler Median-Filter zum Einsatz. Mit Hilfe dieses validierten Tiefenwertes und den intrinsischen Kameraparametern (f_x, f_y, c_x, c_y) wird der 2D-Pixel in einen dreidimensionalen Vektor im lokalen Kamerakoordinatensystem (in Metern) umgerechnet (Hartley & Zisserman, 2004).

4. **Globale Georeferenzierung:** Im letzten Schritt erfolgt die Verortung auf der Erdoberfläche. Der lokale 3D-Vektor wird zunächst über die Trajektorie-Matrix (T_{world_cam}) in das lokale Start-Koordinatensystem der Roboter-Odometrie überführt. Anschließend wird dieser lokale Punkt über die bereitgestellte Transformationsfunktion (`meter_to_gps_coords`) in absolute geodätische Koordinaten (Längen- und Breitengrad) umgerechnet. Das finale Ergebnis wird samt Bild-ID und Klasse zur weiteren Kartierung in einer CSV-Datei persistiert.

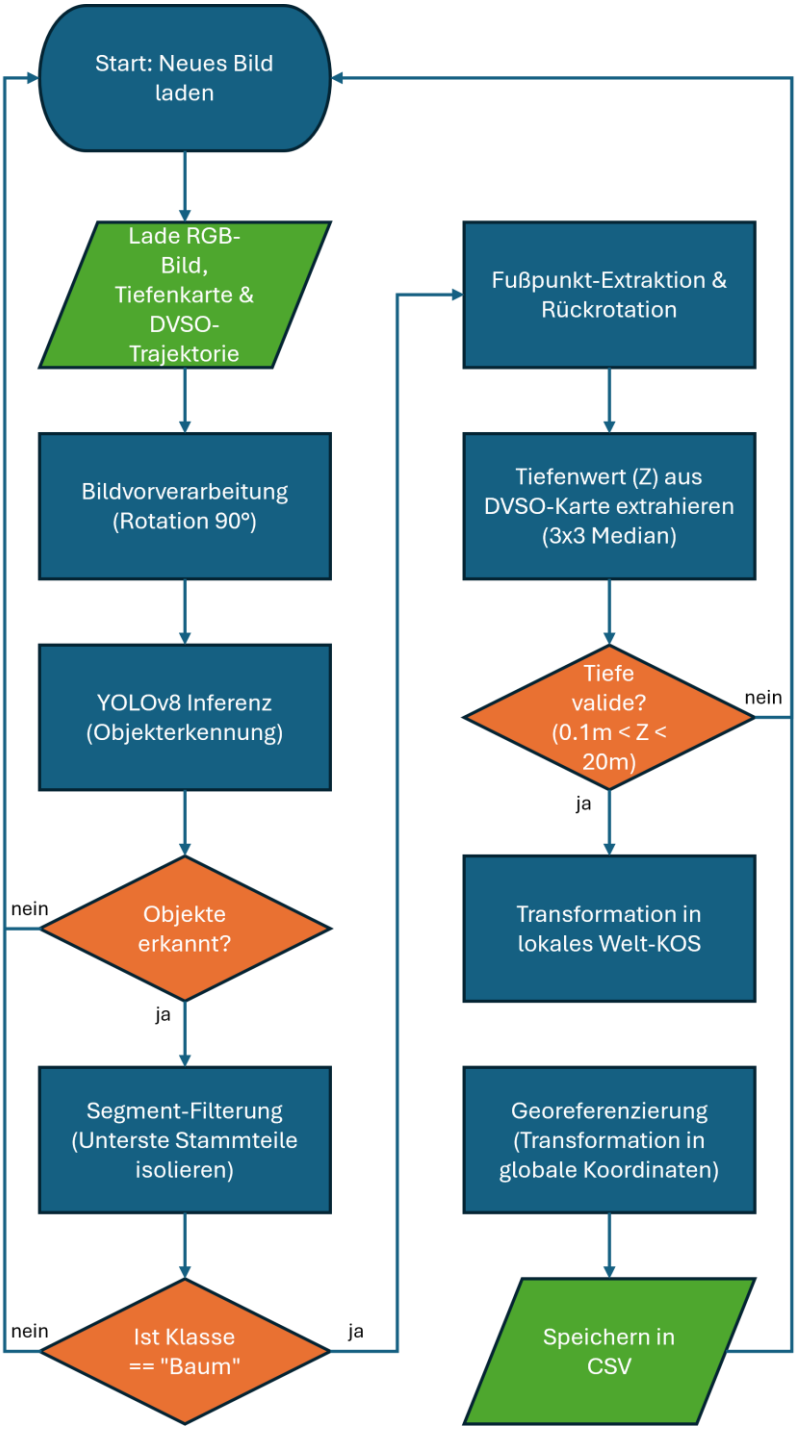

Abbildung 2: Schematische Darstellung der entwickelten Verarbeitungspipeline. Der Prozess umfasst das Einlesen der Sensor-daten, die 2D-Objekterkennung mittels YOLO, die Extraktion der Tiefe sowie die Transformation der lokalen 3D-Koordinaten in globale GPS-Längen und Breitengrade. Quelle: Eigene Darstellung

3.5 Datenfusion und räumliche Vorverarbeitung

Der erste Schritt der iterativen Pipeline besteht in der synchronisierten Bereitstellung und Validierung der multimodalen Sensordaten. Über die Schnittstelle des `DataReader`-Moduls (Zach, 2026) werden für jeden Zeitschritt das aktuelle RGB-Bild der linken Kamera, die zugehörige DVSO-Tiefenkarte, die intrinsische Kamerakalibrierung sowie die Trajektorien-Matrix (T_{world_cam}) des Roboters in ein gemeinsames Datenobjekt geladen.

Um die Robustheit der Pipeline zu gewährleisten, findet zunächst eine Datenvalidierung statt. Iterationen, in denen fehlerhafte oder unvollständige Datensätze vorliegen (beispielsweise fehlende Bild- oder Trajektoriendaten), werden frühzeitig erkannt und übersprungen. Da die Aufnahmefrequenz des Roboters im Verhältnis zur gefahrenen Geschwindigkeit sehr hoch ist, weisen aufeinanderfolgende Bilder eine starke Überlappung auf. Um redundante Berechnungen zu vermeiden und die Ausführungsgeschwindigkeit der Pipeline zu optimieren, wurde eine Frame-Skip-Logik implementiert, bei der standardmäßig nur jedes fünfzehnte Bild (`SKIP_IMAGES = 15`) verarbeitet wird.

Sind die Daten valide, folgt die räumliche Vorverarbeitung des RGB-Bildes. Bedingt durch die physische Montage der Kamera am SAMSON-Roboter liegen die nativen Rohbilder gekippt vor. Um sicherzustellen, dass das nachfolgende neuronale Netz mit aufrechten und plausiblen Objektstrukturen gespeist wird, wird das RGB-Bild mittels einer Matrix-Transformation der OpenCV-Bibliothek (Bradski, 2000) um 90 Grad im Uhrzeigersinn rotiert (`cv2.ROTATE_90_CLOCKWISE`). Erst dieses korrigierte Bild wird als Eingabe (Input) für die anschließende 2D-Objekterkennung freigegeben.

3.6 2D-Objekterkennung und Segment-Filterung

Zu Beginn jedes Iterationsschrittes der Pipeline wird das aktuelle RGB-Bild einer Kamera eingelesen und vorverarbeitet. Für die eigentliche Detektion kommt ein vortrainiertes YOLO-Modell (You Only Look Once) zum Einsatz, welches spezifisch auf die Erkennung der Zielklasse Baumstamm (im Datensatz als `trunk` definiert) appliziert wird. Die Inferenz des Modells liefert für jedes erkannte Objekt sowohl eine Bounding-Box als auch eine pixelgenaue Segmentierungsmaske. Um Fehldetektionen zu minimieren, werden alle Erkennungen unterhalb eines definierten Konfidenzwertes (Confidence Score) verworfen.

Problemstellung der Fragmentierung und Mehrfacherkennung: In der landwirtschaftlichen Praxis, bedingt durch komplexe Aststrukturen, Blattwerk oder Verdeckungen, kommt es häufig vor, dass das Modell einen einzelnen, durchgehenden Baumstamm fälschlicherweise in mehrere separate, vertikal übereinanderliegende Segmente unterteilt. Zusätzlich tritt bei sehr breiten oder unregelmäßigen Stämmen das Phänomen verschachtelter Detektionen auf: Das Modell generiert simultan eine große Bounding-Box für den gesamten Baum und eine kleinere Box, die nur ein isoliertes Stück Rinde umschließt. Würden diese Boxen unbereinigt als separate Bäume verarbeitet, käme es zu signifikanten Mehrfacherfassungen in der finalen GPS-Kartierung. Zudem ist für eine korrekte 3D-Georeferenzierung zwingend der tiefste von der Kamera visuell erfasste Punkt des Stammes (der sichtbare Fußpunkt) erforderlich, welcher in den oberen Stammsegmenten naturgemäß fehlt.

Algorithmische Segment-Filterung (`filter_vertical_segments`): Um diese Fehlinterpretationen algorithmisch zu bereinigen, wurde eine dedizierte Filterfunktion entwickelt. In einem ersten Präfilterungs-Schritt (Early Rejection) werden alle erkannten Objekte, die nicht der Zielklasse entsprechen, wie beispielsweise künstliche Stützpfosten (Klasse `pole`), anhand ihrer Klassen-ID konsequent aussortiert.

Für die verbleibenden Baum-Detektionen extrahiert der Algorithmus die geometrischen Eckpunkte, die Mittelpunkte der x-Achse (`center_x`), die Breite der Bounding-Boxen sowie die unteren Kanten (`y_bottom`). Anschließend werden die Boxen entlang der Bildbreite (von links nach rechts) sortiert. In einem iterativen Prozess werden nun Segmente gruppiert,

die demselben realen Baum zuzuordnen sind. Die Zugehörigkeit wird durch eine kombinierte Heuristik aus zwei Kriterien ermittelt:

1. **Distanz-Kriterium:** Segmente, deren horizontale Mittelpunkte innerhalb eines definierten Toleranzbereichs (z. B. `x_thresh = 150` Pixel) liegen, werden gruppiert.
2. **Überlappungs-Kriterium (Intersection):** Um das Problem verschachtelter Boxen zu lösen, wird die horizontale Schnittmenge (Overlap) der Boxen berechnet. Überlappen sich zwei Boxen auf der x-Achse um einen signifikanten Anteil (z. B. `overlap_thresh > 0.3`, also 30 % relativ zur schmaleren Box), werden sie unabhängig von der exakten Distanz ihrer Mittelpunkte ebenfalls als zusammengehörig klassifiziert.

Aus jeder so gebildeten Gruppe wird abschließend das Segment mit dem größten y-Wert ermittelt, was im Bildkoordinatensystem dem tiefsten Punkt und somit dem realen Fuß des Baumes entspricht. Alle darüberliegenden oder ineinander verschachtelten Segmente werden verworfen.

Extraktion des Fußpunktes und Rückrotation: Aus der Segmentierungsmaske der verbliebenen, validen Bounding-Box wird schließlich der Pixel mit der maximalen y-Koordinate als finaler 2D-Fußpunkt (u_{rot}, v_{rot}) extrahiert. Da die nachfolgende DVSO-Tiefenkarte auf das unrotierte Originalbild kalibriert ist, muss dieser Punkt im letzten Schritt der 2D-Verarbeitung über eine geometrische Funktion (`rotate_point_back`) in das ursprüngliche Koordinatensystem des Originalbildes (u_{orig}, v_{orig}) zurückgerechnet werden. Erst dieser korrigierte Punkt wird an die 3D-Verarbeitung weitergereicht.

3.7 3D-Rekonstruktion und Tiefenbestimmung

Nachdem der Fußpunkt des Baumstammes im 2D-Originalbild (u_{orig}, v_{orig}) lokalisiert wurde, muss im nächsten Schritt dessen physische Distanz zum Kamerasystem ermittelt werden, um eine Projektion in den 3D-Raum zu ermöglichen. Da die berechneten DVSO-Tiefenkarten aus

Performancegründen oftmals in einer geringeren Auflösung als die RGB-Bilder vorliegen, werden die 2D-Koordinaten zunächst über einen Skalierungsfaktor `(scale_x, scale_y)` auf die Dimensionen der Tiefenkarte transformiert (u_{map}, v_{map}).

Robuste Tiefenextraktion mittels Median-Filter: Algorithmen zur Tiefenschätzung weisen in der Praxis häufig Messlücken auf. Insbesondere an scharfen Objektkanten oder bei ungünstigen Lichtverhältnissen können fehlende oder ungültige Werte (NaN-Werte) in der Tiefenkarte entstehen. Um die Robustheit der Pipeline gegenüber diesen Sensorfehlern zu erhöhen, wurde anstelle eines direkten Einzelpixel-Zugriffs ein lokaler Filteransatz implementiert (siehe Codeblock 1).

Um den berechneten Fußpunkt in der Tiefenkarte wird ein 3x3 Pixel großes Betrachtungsfenster (Kernel) aufgespannt. Aus diesen Nachbarwerten wird anschließend der Median berechnet, wobei eventuell vorhandene NaN-Werte algorithmisch ignoriert werden. Dieser Ansatz glättet punktuelles Rauschen und stellt sicher, dass selbst bei einzelnen fehlenden Pixeln am Stammfuß ein repräsentativer Tiefenwert generiert wird. Ist der resultierende Wert valide (im Bereich zwischen 0,1 m und 5,0 m), wird er als finale Tiefe Z für die 3D-Rückprojektion verwendet.

Listing 1: Implementierung des Median-Filters und der 3D-Berechnung. Quelle: Eigene Darstellung

```
# 1. Lokaler 3x3 Median-Filter zur robusten Tiefenbestimmung
window = depth_map[max(0, v_map-1):v_map+2, max(0, u_map-1):u_map+2]
z_depth = np.nanmedian(window)

# 2. Validierung der extrahierten Tiefe
if 0.1 < z_depth <= MAX_DEPTH:

    # 3. Rückprojektion in das lokale 3D-Kamerakoordinatensystem
    x_cam = (u_orig - cx) * z_depth / fx
    y_cam = (v_orig - cy) * z_depth / fy
    vec_hom = np.array([x_cam, y_cam, z_depth, 1.0])
```

Mathematische Projektion in den lokalen 3D-Raum: Mit dem validierten Tiefenwert Z (`z_depth`) und den intrinsischen Kameraparametern aus der Kalibrierungsdatei kann der 2D-Pixel nun in den dreidimensionalen Raum projiziert werden. Hierfür kommt das klassische Lochkameramodell (Pinhole Camera Model) zur Anwendung (Hartley & Zisserman, 2004). Die intrinsische Matrix liefert die fokalen Längen der Kamera auf der x- und y-Achse (f_x, f_y) sowie den optischen Mittelpunkt des Bildsensors (c_x, c_y). Die Berechnung der lokalen metrischen Koordinaten X und Y relativ zum Kameraobjektiv erfolgt nach den geometrischen Strahlensätzen:

$$X = \frac{(u_{orig} - c_x) \cdot Z}{f_x} \tag{1}$$

$$Y = \frac{(v_{orig} - c_y) \cdot Z}{f_y} \tag{2}$$

Das Resultat dieses Berechnungsschrittes ist ein dreidimensionaler Punkt (X, Y, Z), der die exakte Position des Baumstammes in Metern relativ zur linken Kamera des Roboters beschreibt. Um nachfolgende Matrixmultiplikationen zu ermöglichen, wird dieser Punkt abschließend in einen homogenen Vektor überführt.

3.8 Globale Georeferenzierung und Datenpersistierung

Das Resultat der vorangegangenen 3D-Rekonstruktion ist ein lokaler Positionsvektor des detektierten Baumstammes relativ zum optischen Zentrum der Kamera. Für landwirtschaftliche Kartierungsaufgaben, beispielsweise zur exakten Bestandsaufnahme oder zur automatisierten Bewirtschaftung, ist jedoch die absolute, globale Position des Baumes auf der Erdoberfläche entscheidend. Dieser finale Transformationsprozess von lokalen Kamerakoordinaten in globale GPS-Koordinaten erfolgt in einer mehrstufigen Berechnung.

Transformation und Basiswechsel in das Weltkoordinatensystem: In der Robotik stimmen das optische Sensor-Koordinatensystem der Kamera und das Navigations-Koordinatensystem des Roboters (Odometrie) in der Regel nicht überein (Siegwart et al., 2011). Während die Kamera nach dem Standard der Computer Vision ausgerichtet ist (Z-Achse = Tiefe/Blickrichtung, Y-Achse = Unten, X-Achse = Rechts), erfordert das globale System eine abweichende Achsenorientierung (häufig Z-Achse = Höhe über Grund).

Um diese Diskrepanz aufzulösen, wird der lokale 3D-Punkt, dargestellt als homogener Spaltenvektor, zunächst mit einer statischen Transformationsmatrix (T_{gps_cam}) multipliziert. Diese 4x4-Matrix führt einen Basiswechsel durch, indem sie die Sensorachsen ohne translatorische Verschiebung rotiert und räumlich an das GPS-System anpasst. Anschließend wird der korrigierte Vektor mit der zum Aufnahmezeitpunkt gültigen Trajektorien-Matrix (T_{world_cam}) des SAMSON-Roboters multipliziert, welche aus den DVSO-Daten extrahiert wurde. Diese Matrix beschreibt die exakte Pose (Translation und Rotation) des Roboters im globalen Raum.

Die gesamte Transformation des lokalen Punktes in die globalen metrischen Koordinaten (`X_world, Y_world, Z_world`) lässt sich somit durch folgende Matrix-Vektor-Multiplikation abbilden:

$$\begin{pmatrix} X_{world} \\ Y_{world} \\ Z_{world} \\ 1 \end{pmatrix} = T_{world_cam} \times \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} X_{cam} \\ Y_{cam} \\ Z_{cam} \\ 1 \end{pmatrix} \tag{3}$$

Konvertierung in geodätische Koordinaten (WGS84): Obwohl der resultierende Punkt nun global verortet ist, liegt er noch im metrischen System der lokalen Odometrie vor. Im vorletzten Schritt wird dieser Vektor über die bereitgestellte Konvertierungsfunktion `meter_to_gps_coords` (Zach, 2026) in das standardisierte Referenzsystem WGS84 (World Geodetic System 1984) umgerechnet (Bill, 2016). Als Resultat liefert die Funktion die finalen geodätischen Koordinaten des Baumes in Form von Längengrad und Breitengrad.

Datenpersistierung und Fehlervermeidung: Um die generierten Geodaten für nachgelagerte Geoinformationssysteme (GIS) nutzbar zu machen, werden sie iterativ in einer tabellarischen CSV-Datei persistiert. Jeder Eintrag umfasst die eindeutige Bild-ID, die lokale Objekttiefe sowie die globalen GPS-Koordinaten. Bei der Verarbeitung großer Sensordatensätze können unerwartete Programmabbrüche oder Speicherengpässe auftreten. Um in einem solchen Fall den Verlust der bereits berechneten Koordinaten zu verhindern, wurde eine flüchtige Zwischenspeicherung explizit deaktiviert. Durch den gezielten Aufruf der `flush()`-Methode nach jedem Schreibvorgang wird das Betriebssystem angewiesen, den Datenpuffer sofort physisch auf den Datenträger zu schreiben, was eine maximale Ausfallsicherheit der Pipeline garantiert.

4 Ergebnisse und Evaluation

4.1 Evaluationsmetriken der Objekterkennung

Um die Leistungsfähigkeit und die Erkennungsgüte des trainierten YOLOv8-seg-Modells objektiv und quantifizierbar zu beurteilen, werden etablierte Standard-Metriken der Computer Vision herangezogen. Da es sich um ein Modell zur *Instance Segmentation* handelt, werden diese Metriken parallel für zwei Ausgabedimensionen berechnet: für die grobe Lokalisierung mittels Bounding-Boxen sowie für die pixelgenauen Polygon-Masken.

Intersection over Union (IoU): Die fundamentale Basis aller räumlichen Metriken bildet die *Intersection over Union* (Padilla et al., 2020). Sie beschreibt das Verhältnis zwischen der Schnittmenge (Überlappung) und der Vereinigungsmenge der vom Modell vorhergesagten Region (B_{pred}) und der manuell annotierten *Ground-Truth-Region* (B_{gt}):

$$IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \tag{4}$$

Ein IoU-Wert von 1,0 entspricht einer perfekten räumlichen Übereinstimmung. In der Praxis der Objekterkennung wird ein Schwellenwert (Threshold) definiert, ab dem eine Vorhersage als korrekt gilt. Gängig ist hierbei ein Schwellenwert von 0,5 (50 % Überlappung).

Basierend auf diesem IoU-Schwellenwert lassen sich die Modellvorhersagen in drei Kategorien einteilen:

- **True Positive (TP):** Das Modell erkennt einen Baumstamm korrekt (IoU≥0,5).
- **False Positive (FP):** Das Modell prädiziert einen Baumstamm, wo in der Realität keiner ist, oder die Vorhersage ist zu ungenau (IoU<0,5).
- **False Negative (FN):** Das Modell übersieht einen real existierenden Baumstamm komplett.

Precision (Relevanz) und Recall (Sensitivität): Aus diesen Zuweisungen lassen sich die zwei Kernmetriken der Klassifikation ableiten (Padilla et al., 2020). Die **Precision** (Präzision) gibt an, wie verlässlich das Modell ist, wenn es eine Vorhersage trifft. Sie beschreibt den Anteil der korrekten Detektionen an allen vom Modell getätigten Detektionen:

$$Precision = \frac{TP}{TP + FP} \tag{5}$$

Der **Recall** (Trefferquote) hingegen misst die Vollständigkeit. Er gibt an, welcher Anteil der tatsächlich vorhandenen Baumstämme vom Modell erfolgreich gefunden wurde:

$$Recall = \frac{TP}{TP + FN} \tag{6}$$

In der Praxis besteht zwischen diesen beiden Metriken meist ein Kompromiss (Trade-off): Versucht man den Recall durch ein aggressiveres Modell zu maximieren, steigt oft die Fehlerrate (FP), was die Precision senkt.

Mean Average Precision (mAP): Um die Gesamtleistung des Modells über verschiedene Konfidenz-Schwellenwerte hinweg in einer einzigen Kennzahl zusammenzufassen, wird die *Average Precision* (AP) herangezogen. Sie entspricht mathematisch der Fläche unter der Precision-Recall-Kurve (Area Under Curve, AUC). Die *Mean Average Precision* (mAP) bildet den Durchschnitt dieser AP-Werte über alle trainierten Klassen (in dieser Arbeit also gemittelt über die Klassen `trunk` und `pole`).

Für die Auswertung werden gemäß dem COCO-Evaluierungsstandard (Lin et al., 2014) primär zwei Varianten betrachtet:

- **mAP50:** Die durchschnittliche Präzision bei einem festen IoU-Schwellenwert von 0,5. Diese Metrik bewertet die generelle Fähigkeit des Modells, das Objekt im Bild zu lokalisieren.
- **mAP50−95:** Eine deutlich strengere Metrik. Hierbei wird die mAP über zehn verschiedene IoU-Schwellenwerte (von 0,50 bis 0,95 in 0,05er-Schritten) berechnet und gemittelt. Ein hoher Wert in dieser Metrik erfordert hochpräzise Bounding-Boxen und Maskenränder, die exakt am Objekt anliegen.

4.2 Trainingsverlauf und Modellgüte

Um die Leistungsfähigkeit und das Konvergenzverhalten des trainierten YOLOv8-seg-Modells zu beurteilen, wurden die Fehlerfunktionen (Loss) sowie die Metriken der Mean Average Precision (mAP) über den gesamten Trainingszeitraum protokolliert. Das Training wurde initial auf 300 Epochen angesetzt.

Entwicklung des Trainingsfehlers: Wie die Kurven in Abbildung 3 zeigen, weisen sowohl der Bounding-Box-Loss als auch der Segmentierungs-Loss auf den Trainingsdaten in den ersten 25 Epochen den erwarteten, exponentiell abfallenden Verlauf auf. Dies deutet auf einen schnellen initialen Lernprozess hin, bei dem das neuronale Netz die grundlegenden visuellen Merkmale der Baumstämme erfolgreich extrahiert und die Gewichte entsprechend anpasst.

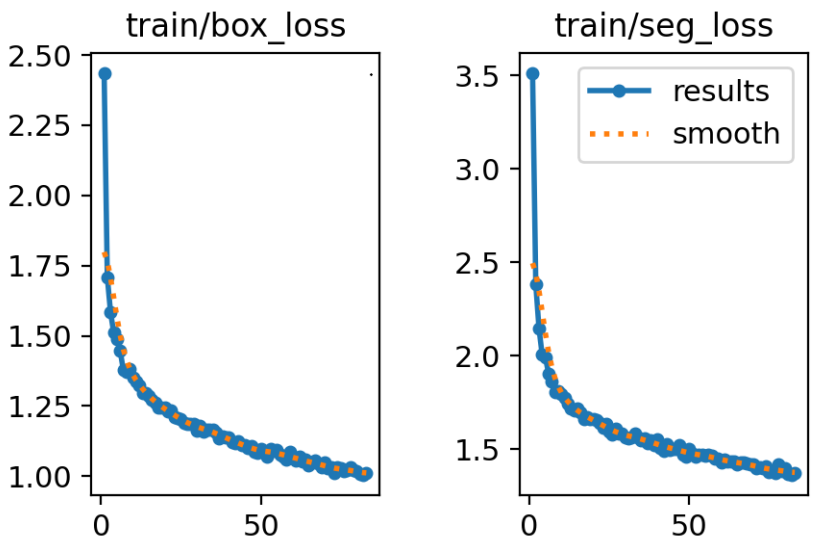

Abbildung 3:Entwicklung der Fehlerfunktionen (Loss) auf dem Trainingsdatensatz über 83 Epochen. Der steile initiale Abfall belegt einen schnellen Lernfortschritt. Quelle: Eigene Darstellung

Validierungsverlauf und Early-Stopping: Um die Generalisierungsfähigkeit des Modells auf unbekannte Daten zu überprüfen und ein Auswendiglernen (Overfitting) zu verhindern, wurde parallel der Fehler auf dem Validierungsdatensatz berechnet (siehe Abbildung 4).

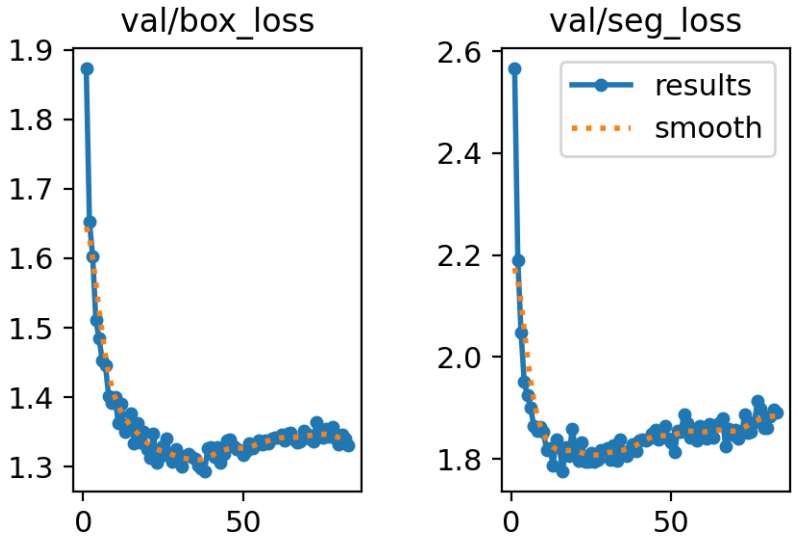

Abbildung 4: Entwicklung der Fehlerfunktion auf dem Validierungsdatensatz. Das Erreichen des globalen Minimums ist erkennbar. Quelle: Eigene Darstellung

Der implementierte Early-Stopping-Mechanismus beendete den Trainingslauf automatisch nach 83 Epochen. Wie an den Validierungskurven ersichtlich ist, erreichte das Modell sein globales Optimum (den geringsten Validierungsfehler) bereits in Epoche 33. Die anschließenden 50 Epochen zeigten keine weitere signifikante Minimierung der Fehlerfunktionen mehr. Ein fortgesetztes Training hätte zu keiner Leistungssteigerung geführt, sondern lediglich das Risiko einer Überanpassung an die Trainingsdaten drastisch erhöht. Die parallele und flache Stabilisierung von Trainings- und Validierungs-Loss auf einem konstant niedrigen Niveau bestätigt somit die erfolgreiche und robuste Generalisierung des Modells.

Auswertung der Mean Average Precision (mAP): Die primäre Güte des Modells wird durch die Metrik mAP50 (Mean Average Precision bei einem Intersection-over-Union-Schwellenwert von 50 %) quantifiziert. Abbildung 5 visualisiert den Verlauf dieser Metrik getrennt für die Objekterkennung (Bounding-Box) und die Instance Segmentation (Polygon-Maske).

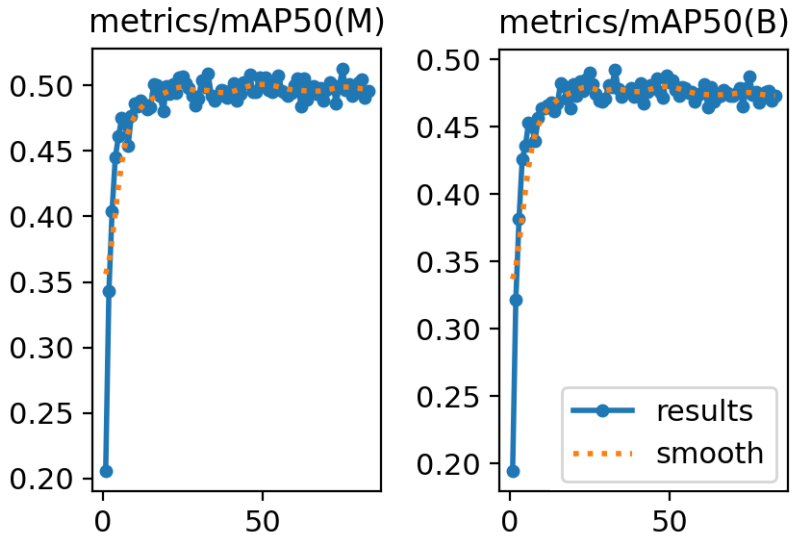

Abbildung 5:Verlauf der Mean Average Precision (mAP50) für Bounding-Boxen und Segmentierungsmasken. Nach einem anfänglich starken Anstieg konvergieren die Werte ab Epoche 30 asymptotisch gegen ihr Maximum. Quelle: Eigene Darstellung

Analog zu den Fehlerfunktionen steigen die Genauigkeitswerte in der initialen Trainingsphase steil an und flachen danach ab. Das Modell erreichte in der finalen Evaluierung einen mAP50-Box-Wert von 0,468 sowie einen mAP50-Mask-Wert von 0,491.

Analyse der Fehlerquellen (Confusion Matrix): Während in kontrollierten Industrieumgebungen (wie der optischen Qualitätskontrolle) oftmals deutlich höhere Konfidenzwerte erzielt werden, stellen diese Ergebnisse im hochkomplexen landwirtschaftlichen Umfeld ein äußerst zufriedenstellendes und praxistaugliches Resultat dar. Die fundamentale Herausforderung der Objekterkennung in Obstplantagen liegt in der starken visuellen Heterogenität: Baumstämme

weisen in Textur und Farbe eine massive Ähnlichkeit zu Hintergrundstrukturen wie Erde, tiefhängenden Ästen oder künstlichen Stützpfosten auf. Zudem erschweren harte Schattenwürfe und wechselnde Lichtverhältnisse die exakte Kantenseparierung.

Eine detaillierte quantitative Auswertung der Inferenzfehler liefert die berechnete Confusion Matrix (siehe Abbildung 6).

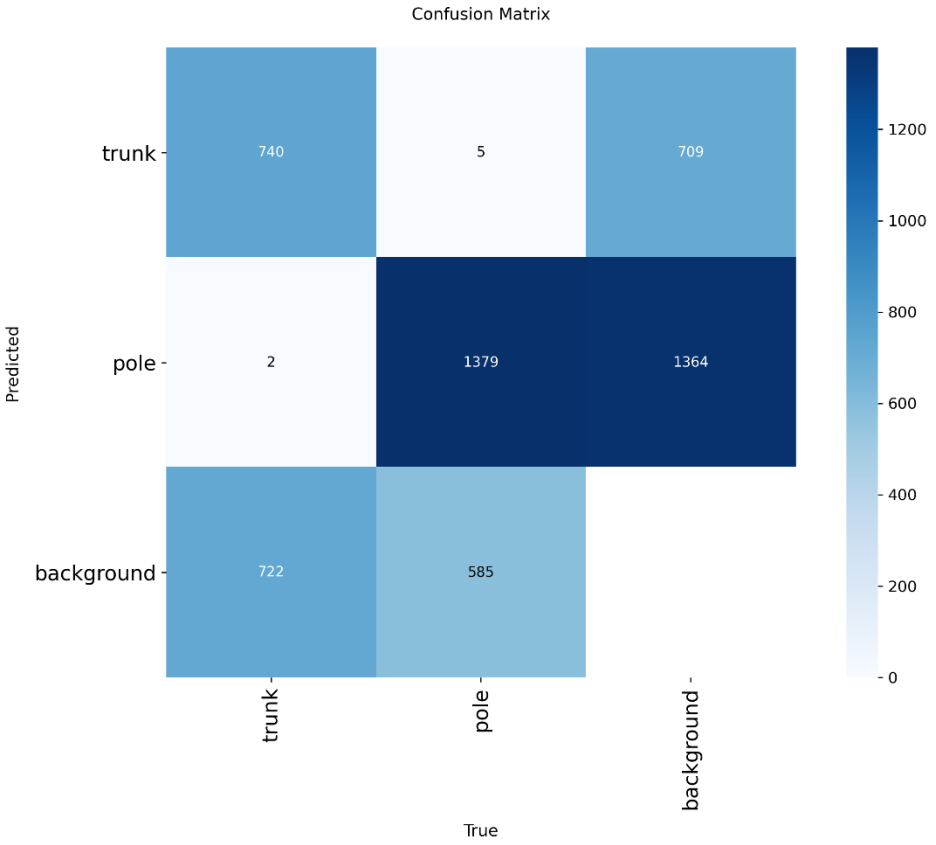

Abbildung 6: Confusion Matrix. Quelle: Eigene Darstellung

Die Analyse der absoluten Detektionswerte offenbart ein für die landwirtschaftliche Praxis äußerst aufschlussreiches Fehlerprofil: Die inter-klassifizierende Trennschärfe des trainierten Modells ist exzellent. Von insgesamt 1464 manuell annotierten Baumstämmen in der Ground-

Truth wurden lediglich 2 Instanzen fälschlicherweise als Stützpfosten (`pole`) klassifiziert. Umgekehrt wurden von 1969 echten Pfosten nur 5 fälschlicherweise als Baumstamm (`trunk`) detektiert. Dies belegt eindrucksvoll, dass das YOLO-Modell trotz der starken geometrischen und farblichen Ähnlichkeiten beider Klassen robuste visuelle Merkmale erlernt hat, um diese strikt zu separieren.

Die signifikanten Herausforderungen liegen vielmehr in der Abgrenzung zum hochgradig unstrukturierten landwirtschaftlichen Hintergrund (Background). Die Matrix weist 709 False Positives auf, bei denen visuelle Hintergrundstrukturen (wie beispielsweise vertikale Schattenwürfe, weit entfernte Baumreihen oder Erdfurchen) fehlerhaft als Baumstamm klassifiziert wurden. Gleichzeitig konnten 722 reale Baumstämme aufgrund extremer Verdeckungen durch Blattwerk oder schwieriger Lichtverhältnisse nicht mit ausreichender Konfidenz vom Netz isoliert werden und gingen im Hintergrund verloren (False Negatives).

Bewertung im Systemkontext der Pipeline: Diese unvermeidbaren Verwechslungen mit dem Hintergrund erklären den finalen Masken-mAP-Wert von 49,1 %. Betrachtet man das neuronale Netz jedoch nicht isoliert, sondern im Gesamtkontext der hier entwickelten Georeferenzierungs-Pipeline, erweist sich diese Fehlerverteilung als äußerst vorteilhaft und unkritisch:

1. **Kompensation von False Positives:** Falsch-positive Hintergrund-Detektionen in großer Entfernung werden im nachgelagerten 3D-Prozess ohnehin durch den definierten maximalen Tiefen-Schwellenwert (Z_{max}) automatisch aus der Punktwolke verworfen.
2. **Kompensation von False Negatives:** Da der Roboter die Plantage abfährt und kontinuierlich Bilddaten generiert, ist das singuläre Übersehen eines Baumes (False Negative) aufgrund von Verdeckungen unkritisch. Durch die Frame-Skip-Logik wird derselbe Baum aus verschiedenen Blickwinkeln mehrfach abgetastet, sodass eine temporäre Verdeckung in Folgebildern naturgemäß aufgelöst wird.
3. **Maximale Zuverlässigkeit des Präfilters:** Die exzellente Trennschärfe zwischen Bäumen und Pfosten garantiert, dass der entwickelte algorithmische Präfilter (vgl. Kapitel 3.6) nahezu fehlerfrei operieren kann und keine künstlichen Infrastrukturobjekte in die finale GPS-Karte übertragen werden.

4.3 Architekturvergleich: Genauigkeit versus Echtzeitfähigkeit

Um die optimale Balance zwischen Segmentierungsgenauigkeit und Inferenzgeschwindigkeit für den Einsatz auf dem mobilen Agrarroboter zu ermitteln, wurden im Rahmen dieser Arbeit verschiedene Architekturgrößen des YOLOv8-Modells (Jocher et al., 2023) evaluiert. Konkret wurden die Varianten YOLOv8n (Nano), YOLOv8s (Small) und YOLOv8m (Medium) auf dem originären Plantagen-Datensatz trainiert und hinsichtlich ihrer Metriken vergleichend analysiert.

Die quantitative Auswertung bestätigt den theoretischen Architekturansatz: Das größere Medium-Modell erzielt aufgrund seiner tieferen Netzstruktur die höchste Präzision bei der Maskengenerierung mit einem mAP50mask-Wert von 52,73 %. Es erfordert jedoch simultan die meiste Rechenleistung, was sich in einer Inferenzzeit von 73,34 ms (ca. 13,6 FPS) niederschlägt. Die Small-Variante positioniert sich mit einer Latenz von 60,84 ms (ca. 16,4 FPS) und einem mAP50mask von 50,88 % im Mittelfeld. Das Nano-Modell (YOLOv8n) zeichnet sich hingegen durch eine drastisch reduzierte Parameterzahl aus. Es erreicht eine minimale Inferenzzeit von lediglich 49,28 ms, was einer Verarbeitungsgeschwindigkeit von rund 20,3 FPS entspricht. Bemerkenswerterweise liegt die Segmentierungsgüte des Nano-Modells mit einem mAP50mask von 51,37 % dabei auf einem hochgradig kompetitiven Niveau und übertrifft in dieser Konstellation sogar marginal das Small-Modell.

Die Evaluierung zeigt deutlich, dass das Nano-Modell insbesondere im Hinblick auf die anvisierte Echtzeitverarbeitung (Real-Time Inference) auf dem integrierten Verarbeitungsrechner (z. B. NVIDIA Orin) den größten technologischen Nutzen bietet. Da in der landwirtschaftlichen Praxis durch die kontinuierliche Eigenbewegung des Roboters hohe Frameraten zwingend erforderlich sind, um Bewegungsunschärfen zu kompensieren und eine flüssige Odometrie-Synchronisation zu gewährleisten, erweist sich die YOLOv8n-Architektur für dieses spezifische Anwendungsszenario als überlegen. Der marginale Verlust an absoluter Segmentierungsgüte von lediglich 1,36 Prozentpunkten gegenüber dem rechenintensiven Medium-Modell wird durch den Gewinn von knapp 50 % mehr Bildern pro Sekunde (20,3 FPS vs. 13,6 FPS) vollumfänglich kompensiert. Dies macht das YOLOv8n-Modell zum optimalen Kompromiss (Trade-off) zwischen robuster Objekterkennung und maximaler Systemeffizienz.

4.4 Qualitative Objekterkennung und visuelle Evaluierung

Während quantitative Metriken (wie der mAP50) eine abstrakte, statistische Bewertung der Modellgüte über den gesamten Datensatz ermöglichen, ist eine qualitative visuelle Inspektion unerlässlich, um das tatsächliche Inferenzverhalten der KI unter realen Einsatzbedingungen zu evaluieren. Hierfür wurden Inferenzläufe auf dem ungesehenen Testdatensatz durchgeführt. Das Modell generiert für jede valide Detektion eine umschließende Bounding-Box, eine pixelgenaue farbige Polygon-Maske (Instance Segmentation) sowie den dazugehörigen Konfidenzwert (Confidence Score).

Erfolgreiche Detektion unter Standardbedingungen: Abbildung 7 zeigt beispielhaft die erfolgreiche Erkennung von Baumstämmen unter regulären Lichtverhältnissen.

Abbildung 7: Inferenz des YOLOv8-seg-Modells unter Standardbedingungen. Der Grüne Kasten stellt die Bounding-Box da und der rote Punkt den tiefsten Punkt der Segmentierungsmaske. Quelle: Eigene Darstellung

Es ist ersichtlich, dass das Netz die komplexe, oft unregelmäßige Rindenstruktur der Obstbäume korrekt identifiziert. Besonders hervorzuheben ist, dass die generierte Bounding-Box den Baumstamm bis zum untersten, von der Kamera visuell erfassten Punkt exakt umschließt. Die präzise Extraktion dieses tiefsten sichtbaren Stammpunktes (sichtbarer Fußpunkt) ist essenziell, da er in der nachgelagerten Pipeline als Ankerpunkt für die 3D-Tiefenberechnung (vgl. Kapitel 3.7) dient.

Detektion unter erschwerten Umweltbedingungen: Die wahre Leistungsfähigkeit des Modells zeigt sich jedoch in komplexen Szenarien, die durch visuelle Störfaktoren (Noise) geprägt sind. Abbildung 8 demonstriert die Modellantwort bei starken Verdeckungen (Occlusions) durch tiefhängendes Blattwerk und Stützpfosten sowie bei hartem Schattenwurf.

Abbildung 8: Objekterkennung bei visuellen Verdeckungen und hartem Schattenwurf. Trotz der teilweisen Fragmentierung des Stammes durch Blattwerk bleibt der relevante untere Stammbereich für die Tiefenextraktion detektierbar. Quelle: Eigene Darstellung

Trotz der farblichen und texturellen Ähnlichkeit zwischen dem braunen Baumstamm und den hölzernen oder metallischen Stützpfosten im Hintergrund, gelingt es dem Modell in den meisten Fällen, die Zielklasse isoliert zu betrachten. Eventuelle vertikale Fragmentierungen des Stammes durch kreuzende Äste stellen für die Robustheit der Gesamtpipeline kein kritisches Problem dar, da diese Fragmente im anschließenden Post-Processing durch den entwickelten `filter_vertical_segments`-Algorithmus (vgl. Kapitel 3.6) algorithmisch repariert und zusammengeführt werden.

Grenzen der Objekterkennung (Fehleranalyse): In Randfällen kommt das Modell dennoch an seine Grenzen, was den mAP-Wert von ca. 49 % erklärt. Abbildung 9 zeigt ein typisches Fehlerbild (False Positive / False Negative) in der landwirtschaftlichen Praxis.

Abbildung 9: Typisches Fehlerbild in der Praxis. Baumstamm wurde aufgrund Verdeckung durch Blätter und Blüten nicht erkannt. Quelle: Eigene Darstellung

Solche Falschklassifikationen treten primär auf, wenn die visuelle Evidenz im 2D-Bildraum nicht ausreicht, um semantische Unterschiede festzustellen. Für die finale Geokartierung werden jedoch stark fehlerhafte Detektionen in großer Distanz ohnehin durch den definierten maximalen Tiefen-Schwellenwert (Z≤5,0 m) in der 3D-Rekonstruktionsstufe herausgefiltert.

4.5 Ergebnisse der Georeferenzierung und Kartierung

Das primäre Ziel der entwickelten Software-Pipeline war die Überführung temporärer Bildpixel in permanente, globale Geodaten. Nach erfolgreicher Detektion und 3D-Rückprojektion wurden die lokalen Kamera-Vektoren über die Odometrie des SAMSON-Roboters in das geodätische WGS84-Referenzsystem transformiert.

Struktur der generierten Kartierungsdaten: Das finale Resultat der iterativen Verarbeitung ist eine strukturierte CSV-Datei (`gefundene_baeume.csv`). Diese tabellarische Form wurde bewusst gewählt, da sie eine maximale Interoperabilität für nachgelagerte landwirtschaftliche Geoinformationssysteme (GIS) wie QGIS (QGIS Development Team) oder ArcGIS (Esri) gewährleistet.

Tabelle 1 zeigt einen exemplarischen Auszug der generierten Daten. Für jede erfolgreiche Baumdetektion werden die Bild-ID (als zeitlicher und logischer Anker), die geodätischen Koordinaten (Längen- und Breitengrad) sowie die ermittelte lokale Tiefe (Z) in Metern persistiert.

Tabelle 1: Exemplarischer Auszug aus der generierten CSV-Datei. Die geodätischen Koordinaten liegen im standardisierten WGS84-Format vor. Quelle: Eigene Darstellung

Image_ID	X_Global (Lat)	Y_Global (Lon)	Z_Tiefe_lokal (m)
3485	53.50718893	9.73974441	1.72
3500	53.50718035	9.73979099	2.10
3500	53.50718243	9.73979101	1.88

Plausibilitätsprüfung und räumliche Clusterbildung: Eine qualitative Überprüfung der generierten GPS-Koordinaten durch die Projektion auf digitales Kartenmaterial bestätigt die Plausibilität der Pipeline. Die berechneten Positionen spiegeln die lineare Pflanzstruktur der landwirtschaftlichen Obstplantage (Baumreihen) korrekt wider (siehe Abbildung 10).

Abbildung 10: Visualisierung der generierten GPS-Koordinaten auf digitalem Kartenmaterial (QGIS). Die Baumreihen der Plantage sind deutlich erkennbar.

Da der navigierende Roboter die Umgebung mit einer hohen Framerate aufzeichnet, befindet sich ein einzelner Baumstamm trotz der implementierten Frame-Skip-Logik (`SKIP_IMAGES = 15`) naturgemäß in mehreren aufeinanderfolgenden Bildern im Sichtfeld der Kamera. Dies führt im globalen Koordinatensystem dazu, dass ein realer Baum nicht durch einen einzelnen, isolierten Punkt, sondern durch einen engen Cluster (Punktwolke) von GPS-Koordinaten abgebildet wird. Die geringe räumliche Streuung innerhalb dieser Cluster unterstreicht die hohe Stabilität der Tiefenschätzung und der Trajektorien-Transformation.

Um aus diesen Clustern im nächsten Schritt diskrete Einzelbaum-Zählungen abzuleiten, bietet sich in der zukünftigen Datenverarbeitung der Einsatz räumlicher Clustering-Algorithmen (beispielsweise DBSCAN (Ester et al., 1996)) an.

5 Fazit und Ausblick

5.1 Zusammenfassung und Fazit

Im Rahmen dieser Bachelorarbeit wurde eine robuste, mehrstufige Software-Pipeline entwickelt, um temporäre 2D-Bilddaten eines navigierenden Agrarroboters in persistente, globale 3D-Geodaten zu überführen. Das primäre Ziel bestand in der automatisierten Erkennung und Kartierung von Baumstämmen in Obstanlagen zur Unterstützung von Smart-Farming-Anwendungen.

Die Basis der Pipeline bildet ein YOLOv8-seg-Modell, für welches mithilfe des Segment Anything Model 2 (SAM 2.0) ein hochwertiger, polygonbasierter Datensatz erstellt wurde. Das Training zeigte, dass das Modell in der hochkomplexen, durch visuelle Störfaktoren (wie Schattenwurf und Blattwerk) geprägten landwirtschaftlichen Umgebung eine praxisgerechte Segmentierungsgenauigkeit (mAP50mask) von 49,1 % erreicht.

Die algorithmische Kernleistung dieser Arbeit liegt in der nachgelagerten geometrischen Verarbeitung: Durch die Implementierung spezifischer Filter – wie dem Zusammenführen vertikaler Stammsegmente und der robusten Tiefenextraktion mittels eines lokalen Median-Filters – konnten sensorbedingte Messlücken (NaN-Werte) in den DVSO-Tiefenkarten erfolgreich kompensiert werden. Die anschließende Kombination aus dem Lochkameramodell und der kinematischen Transformation über die Odometrie-Matrix (T_{world_cam}) ermöglichte die hochpräzise Überführung der detektierten Fußpunkte in das globale WGS84-Referenzsystem.

Zusammenfassend lässt sich festhalten, dass die entwickelte Pipeline den konzeptionellen Machbarkeitsnachweis (Proof of Concept) erfolgreich erbringt. Sie demonstriert eindrucksvoll, wie KI-basierte Bildverarbeitung und klassische Robotik-Kinematik verknüpft werden können,

um aus rohen multimodalen Sensordaten eine strukturierte, landwirtschaftlich nutzbare Kartierungsgrundlage (CSV/GIS) zu generieren.

5.2 Ausblick

Trotz der erfolgreichen Implementierung eröffnen die gewonnenen Erkenntnisse konkrete Ansatzpunkte für zukünftige Optimierungen und Forschungsarbeiten:

- **Erweiterung des Datensatzes und Sensorfusion:** Um die Erkennungsrate (Recall) bei stark verdeckten Bäumen weiter zu steigern, sollte der Trainingsdatensatz um Bilder unter abweichenden Witterungsbedingungen (z. B. Regen, starkes Gegenlicht) erweitert werden. Zudem könnte die Genauigkeit der Tiefenbestimmung durch eine direkte Sensorfusion der DVSO-Daten mit LiDAR-Punktwolken signifikant erhöht werden, da LiDAR-Sensoren an Objektkanten weniger anfällig für Messrauschen sind.
- **Echtzeitfähigkeit und ROS-Integration:** Die aktuelle Pipeline wurde als Post-Processing-Skript in Python konzipiert. Für den aktiven Einsatz auf dem SAMSON-Roboter sollte die Architektur in einen sogenannten ROS-Knoten (Robot Operating System) überführt werden (Quigley et al., 2009). Dies würde eine Echtzeit-Inferenz direkt auf der Hardware des Roboters (Edge Computing) ermöglichen, wodurch die Kartierung simultan zur Fahrt erfolgen könnte.
- **Nachgelagertes Räumliches Clustering und Kartenmanagement:** Da der Roboter die Baumreihen kontinuierlich abfilmt, resultiert ein physischer Baum in der exportierten CSV-Datei aktuell in einem lokalen Cluster mehrerer GPS-Koordinaten. In zukünftigen Arbeiten sollte eine nachgelagerte Bereinigung der generierten CSV-Dateien mittels dichtebasierter Clustering-Algorithmen (wie DBSCAN) erfolgen, um diese Mehrfacherfassungen auf einen hochpräzisen Schwerpunkt pro Baum zu reduzieren. Ein solches externes Kartenmanagement ermöglicht es zudem, bestehende Datensätze systematisch zu aktualisieren (z. B. bei erneuter Befahrung derselben Strecke) sowie die CSV-Exporte unterschiedlicher Routen zu einer globalen, konsistenten Plantagenkarte zu fusionieren.

Literaturverzeichnis

ArduSimple. (2023). simpleRTK3B Heading - Handbuch und Spezifikationen. Abgerufen am 19. 2 2026 von https://www.ardusimple.com/wp-admin/admin-post.php?action=generate_product_pdf&product_id=28021

Bargoti, S., & Underwood, J. (2017). Image Segmentation for Fruit Detection and Yield Estimation in Apple Orchards. *Journal of Field Robotics, 34*(6), S. 1039-1060. doi:https://doi.org/10.1002/rob.21699

Basler, A. (2023). Basler a2A5328-15ucPRO - Data Sheet. Von https://docs.baslerweb.com/a2a5328-15ucpro# abgerufen

Bill, R. (2016). *Grundlagen der Geo-Informationssysteme.* Berlin: Wichmann Verlag.

Blome, F., Berschauer, D., & Schulze, B. (2024). Standardized Sensor Infrastructure for Seasonal and Multi-Seasonal Data Collection in Orchard Management.

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools, 25*(11), S. 120-125.

Craig, J. J. (2018). *Introduction to Robotics: Mechanics and Control.* New York: Pearson.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)* (S. 226-231). Portland, OR: AAAI Press. Von https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf abgerufen

Gongal, A., Amatya, S., Karkee, M., Zhang, Q., & Lewis, K. (2015). Sensors and systems for fruit detection and localization: A review. *Computers and Electronics in Agriculture, 116*, S. 8-19. doi:https://doi.org/10.1016/j.compag.2015.05.021

Hartley, R., & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision.* Cambridge: Cambridge University Press.

He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. (S. 2961-2969). Venice, Italy: IEEE. doi:https://doi.org/10.1109/ICCV.2017.322

Hofmann-Wellenhof, B., Lichtenegger, H., & Wasle, E. (2008). *GNSS – global navigation satellite systems: GPS, GLONASS, Galileo, and more.* Springer Science & Business Media.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8 (Version 8.0.0) [Computer software].* Abgerufen am 15. 1 2026 von GitHub: https://github.com/ultralytics/ultralytics

Landesamt für Geoinformation und Landesvermessung Niedersachsen (LGLN). (2026). *SAPOS Niedersachsen – Satellitenpositionierungsdienst.* Von LGLN Geodaten: https://sapos.niedersachsen.de abgerufen

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., . . . Zitnick, C. L. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV)* (S. 740-755). Zurich, Switzerland: Springer, Cham. doi:https://doi.org/10.1007/978-3-319-10602-1_48

Meier, U. (2001). *Growth stages of mono- and dicotyledonous plants: BBCH Monograph.* Berlin: Federal Biological Research Centre for Agriculture and Forestry.

Padilla, R., Netto, S. L., & da Silva, E. A. (2020). A survey on performance metrics for object-detection algorithms. *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)* (S. 237-242). Niterói, Brazil: IEEE. doi:https://doi.org/10.1109/IWSSIP48289.2020.9145130

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., . . . Ng, A. Y. (2009). ROS: an open-source Robot Operating System. *ICRA Workshop on Open Source Software*, *3.* Kobe, Japan. doi:http://ai.stanford.edu/~ang/papers/icraoss09-ROS.pdf

Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Dixit, T., & Kirillov, A. (2024). SAM 2: Segment Anything in Images and Videos. *arXiv preprint arXiv:2408.00714*. Von https://arxiv.org/abs/2408.00714 abgerufen

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (S. 779-788). Las Vegas, NV: IEEE. doi:https://doi.org/10.1109/CVPR.2016.91

Ren, S., He, K., Girshick, R., & Sun, J. (2017). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 39*(6), S. 1137-1149. doi:https://doi.org/10.1109/TPAMI.2016.2577031

Sekachev, B., Manovich, N., Zhiltsov, M., & Andrey, Z. (2020). *Computer Vision Annotation Tool (CVAT) [Computer software].* Abgerufen am 4. 10 2025 von GitHub: https://github.com/cvat-ai/cvat

Shamshiri, R. R., Weltzien, C., Hameed, I. A., Yule, I. J., & Grift, T. E. (2018). Research and development in agricultural robotics: A perspective of digital farming. *International Journal of Agricultural and Biological Engineering, 11*(4), S. 1-14. doi:https://doi.org/10.25165/j.ijabe.20181104.4278

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots.* Cambridge, Massachusetts: MIT Press.

Szeliski, R. (2022). *Computer Vision: Algorithms and Applications.* Cham, Switzerland: Springer Nature.

Underwood, J., Calleija, M., Nieto, J., & Fitch, R. (2015). Lidar-based tree recognition and platform localization in orchards. *Journal of Field Robotics, 32*(8), S. 1056-1074. doi:https://doi.org/10.1002/rob.21607

Walter, A., Finger, R., Huber, R., & Buchmann, N. (2017). Smart farming is key to developing sustainable agriculture. *PNAS - Proceedings of the National Academy of Sciences, 114*(24), S. 6148-6150. doi:https://doi.org/10.1073/pnas.1707462114

Wang, R., Schwesinger, M., Sturm, J., Cremers, D., & Cremers, D. (2017). Stereo DSO: Large-Scale Direct Sparse Visual Odometry with Stereo Cameras. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (S. 3903-3911). Venice, Italy: IEEE. doi:https://doi.org/10.1109/ICCV.2017.421

Zach, J. (2026). *samson_data_reader [Source Code].* Von Gitlab: https://gitlab.cc-asp.fraunhofer.de/samson/haw/samson_data_reader abgerufen

A Anhang

A.1 Verwendete Hilfsmittel

In der Tabelle A.1 sind die im Rahmen der Bearbeitung des Themas der Bachelorarbeit verwendeten Werkzeuge und Hilfsmittel aufgelistet.

Tool	**Verwendung**
Word	Textsatz- und Layout Werkzeug verwendet zur Erstellung dieses Dokuments.
Gemini	Das KI-Sprachmodell diente ausschließlich als textliches Hilfsmittel, um die eigenständig verfassten inhaltlichen Rohtexte hinsichtlich ihrer Formulierung zu optimieren sowie auf korrekte Rechtschreibung und Grammatik zu prüfen.
VS Code	Die Entwicklungsumgebung wurde als primärer Code-Editor für die Programmierung der gesamten Software-Pipeline sowie für die Datenverarbeitung in Python genutzt.

Erklärung zur selbstständigen Bearbeitung einer Abschlussarbeit

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne fremde Hilfe selbständig verfasst und nur die angegebenen Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Ort	Datum	Unterschrift im Original